

HIGH-LEVEL TEST PLAN (UPDATED)

Project: CS160 Team Project - Banking System

Team 2

Last updated: April 27, 2026

1. Purpose and Scope

This document updates the original Part 2 high-level test plan to match the current implementation in this repository.

Current implemented scope includes:

- React web app (no in-repo React Native client)
- FastAPI backend with Supabase-backed persistence
- Customer banking flows: auth/profile/accounts/transfers/bill pay/deposits/transactions/notifications/ATM locator
- Admin flows: account directory + account close (admin delete action) + reporting dashboard with account/customer filters
- Internal job runners for scheduled payments and transfer plans

2. Test Strategy

Testing approach (current-state):

- Frontend unit/component tests with Vitest + React Testing Library
- Contract/normalization tests for API payload mapping
- Backend endpoint and workflow verification via integration/manual testing against Supabase
- Manual exploratory UX testing for end-to-end banking flows
- Regression testing for scheduling/idempotency/error handling paths

Current automated tests in repo:

- ``frontend/src/features/dashboard/DashboardPage.test.tsx``
- ``frontend/src/features/accounts/AccountDetailPage.test.tsx``
- ``frontend/src/features/transfers/TransfersPage.test.tsx``
- ``frontend/src/features/bill-pay/BillPayPage.test.tsx``
- ``frontend/src/lib/bankingContract.test.ts``
- ``frontend/src/lib/format.test.ts``

3. Functional Test Areas

3.1 Auth and Session

Goal:

- Validate login/logout/reset-password/session hydration and protected route behavior.

Expected behavior:

- Authenticated session unlocks `/app/\*` routes.
- Admin role gates `/admin/\*` routes.
- Password reset updates credentials and requires fresh sign-in.
- Session is cleared on backend restart boot-id mismatch (current implementation behavior).

Failure handling expectations:

- Invalid credentials do not authenticate.
- Missing/expired tokens block protected API calls.
- Unauthorized admin access returns 403 and does not render admin content.

Success criteria:

- Correct route gating for user vs admin.
- No leaked auth state after sign-out or backend restart.
- Clear UI errors for auth failures.

3.2 Customer Profile

Goal:

- Ensure profile retrieval/update works with validation.

Expected behavior:

- `/api/me/profile` returns customer record.
- Profile patch enforces phone and ZIP normalization rules.

Failure handling expectations:

- Invalid phone/ZIP rejected with 400.
- Missing profile returns 404 and surfaces cleanly in UI.

Success criteria:

- Profile edits persist and rehydrate correctly.

3.3 Accounts

Goal:

- Validate account creation/list/detail/close and closure eligibility reasons.

Expected behavior:

- Account creation produces unique account/routing identifiers.
- Open accounts are visible in customer account views.
- Close is allowed only for eligible accounts (zero balances, no blocking activity).

Failure handling expectations:

- Close attempts on blocked accounts return structured 409 reasons.
- Nonexistent account returns 404.

Success criteria:

- No partial state changes on failed close.
- Account state transitions are consistent and auditable.

3.4 Internal Transfers

Goal:

- Validate immediate and scheduled internal transfer behavior.

Expected behavior:

- Valid transfers create correct status/result payloads.
- Scheduled plan metadata is stored and runner-processed.
- Ownership checks and account status checks enforced.

Failure handling expectations:

- Insufficient funds and invalid account combinations rejected.
- Failed transfers do not partially debit/credit balances.

Success criteria:

- Balances and transaction states match final outcomes.
- No double-application for equivalent requests.

3.5 Member Transfers

Goal:

- Validate recipient resolution, immediate member transfers, and recurring plans.

Expected behavior:

- Recipient lookup resolves valid users.

- Transfers and plans follow cadence/timezone fields.
- Plan cancel/update/retry endpoints behave deterministically.

Failure handling expectations:

- Invalid recipient or ineligible source account rejected.
- Runner failures do not corrupt account state.

Success criteria:

- Correct state progression for transfer and plan records.

3.6 External Accounts and External Transfers

Goal:

- Validate external account linking and outbound transfer safety.

Expected behavior:

- Manual and link-session completion paths create external accounts.
- External transfers enforce verified/active external account constraints.
- Plans support cancel/update/retry.

Failure handling expectations:

- Invalid routing/account input rejected.
- Duplicate link attempts rejected.
- Failed processing does not debit source balances incorrectly.

Success criteria:

- External transfer state transitions are correct and reproducible.

3.7 Bill Pay

Goal:

- Validate one-time and recurring payment scheduling/execution paths.

Expected behavior:

- Create/list/update/cancel/retry endpoints work with cadence rules.
- Idempotency-Key is required where enforced and prevents duplicate writes.
- Scheduler updates `next\_run\_at` and statuses correctly.

Failure handling expectations:

- Insufficient funds leaves balances unchanged and sets failure reason.

- Duplicate submission with same idempotency key replays prior response.

Success criteria:

- No duplicate executions for the same idempotent request.
- Payment lifecycle remains consistent through retries/cancellations.

3.8 Deposits

Goal:

- Validate deposit upload URL generation and deposit submission lifecycle.

Expected behavior:

- Signed upload URLs are generated for front/back images.
- Deposit submission records status and appears in history/detail views.

Failure handling expectations:

- Invalid account or malformed request rejected.
- Failed deposit flow does not partially mutate balances.

Success criteria:

- Deposit status transitions are consistent and visible in UI.

3.9 Transactions and Notifications

Goal:

- Verify transaction filters and notification read flow.

Expected behavior:

- Transaction list supports account/type/status/limit filtering.
- Notification list returns newest-first and supports mark-as-read.

Failure handling expectations:

- Invalid filters do not crash API/UI.
- Missing notification IDs return clean 404.

Success criteria:

- Consistent filtering behavior and correct unread/read updates.

3.10 ATM Locator

Goal:

- Validate geocode/location ATM search and resilient map fallback behavior.

Expected behavior:

- ATM results returned by query or coordinates.
- Distance/open-now/radius filters work in UI.
- If `VITE\_GOOGLE\_MAPS\_API\_KEY` is absent/invalid, list still works and map degrades gracefully.

Failure handling expectations:

- Invalid address/external API failures show user-friendly errors.
- No UI crash when map script load fails.

Success criteria:

- Deterministic, stable ATM list behavior under valid and degraded conditions.

3.11 Admin Accounts and Reporting

Goal:

- Validate admin account oversight and report generation.

Expected behavior:

- `/api/admin/accounts` lists active (non-closed) accounts.
- Admin delete action closes account through closure RPC and returns reasons when blocked.
- `/api/admin/reports/accounts` supports search/filtering by account/customer attributes (including balance and ZIP/city/state).
- CSV export from admin reporting dashboard matches on-screen dataset.

Failure handling expectations:

- Non-admin access blocked.
- Report filter edge cases (invalid ranges/no results) handled gracefully.
- Close/delete attempts on ineligible accounts return actionable reasons.

Success criteria:

- Admin workflows are role-protected, deterministic, and user-friendly.

3.12 Internal Job Runner Endpoints

Goal:

- Validate protected scheduler endpoints for due plan/payment processing.

Expected behavior:

- Runner secret is required and validated.

- Bounded batch limits enforced.
- Returned processing counts are coherent with queued work.

Failure handling expectations:

- Missing/invalid secret returns 401.
- Missing configured secret returns 503.

Success criteria:

- Scheduled processing executes without unauthorized access.

4. Nonfunctional Testing

Performance:

- Baseline key API calls under local dev load remain responsive.
- Large list views (transactions/report rows) remain usable with filtering/pagination limits.

Security:

- All protected endpoints require bearer auth.
- Admin endpoints require admin role checks.
- Idempotency and server-side validation prevent duplicate side effects.

Reliability:

- Failure paths do not leave partial balance mutations.
- Runner and retry operations are resilient to transient failures.

Observability:

- Error payloads are structured and actionable for UI display.
- Regression checks include account close eligibility reasons and payment/transfer failure reasons.

Compatibility:

- Web validation in modern desktop browsers (Chrome/Safari/Edge baseline).
- Responsive checks for key customer/admin pages.

5. Test Tools and Execution

Frontend:

- ``npm run test``
- ``npm run lint``
- ``npm run build``

Backend:

- API integration checks via local FastAPI + Supabase environment
- Contract verification against `/api/\*` endpoints used by frontend services

Manual regression suite (minimum per release candidate):

1. Auth/login/logout/reset-password
2. Account open/close eligibility and blocked-close reasons
3. Internal/member/external transfer critical paths
4. Bill pay create/update/cancel/retry + idempotency
5. Deposit submit + history/detail
6. ATM search with and without map key
7. Admin account close action + report generation/export
8. Internal job endpoint authorization and execution checks

6. Current Gaps and Next Additions

Recommended next additions to improve confidence:

- Add backend pytest coverage for transfer/payment/deposit/admin routers.
- Add E2E smoke tests for the top user journeys (Playwright/Cypress).
- Add load tests for scheduler and high-volume report queries.
- Add explicit browser matrix evidence in CI artifacts.